



دانشگاه صنعتی شریف
دانشکده مهندسی کامپیوتر

پروژه سوم
سیستم های عامل

پایه سازی سیستم مدیریت استقرار و مدیریت کانتینر (Zocker)

نگارش

گروه ۸

مبینا رشیدی (۴۰۱۱۷۰۵۶۴)

فاطمه روستا (۴۰۱۱۰۵۹۹۱)

محمدنوید آتشین بار (۴۰۲۱۰۵۵۸۱)

کیما علوی (۴۰۲۱۰۶۲۱۸)

استاد

اسدی

بهمن ۱۴۰۴

چکیده

هدف نهایی این پروژه، ارتقای ابزار مدیریت کانتینر جهت پشتیبانی از اجرای چندین کانتینر به صورت daemonless بر اساس استاندارد OCI است. در بخش اول، سیستم باید قابلیت اجرای کانتینرها در پس‌زمینه (background) را با تکیه بر systemd فراهم کند. دستورات مدیریتی مورد نیاز شامل exec، restart، stop و ps است که باید بر روی کانتینرهای در حال اجرا و متوقف شده پیاده‌سازی شوند. مطابق با استاندارد OCI، برای هر کانتینر فایل‌های config.json شامل تنظیمات اجرا و state.json شامل وضعیت و اطلاعات زنده ایجاد می‌گردد. علاوه بر این، برای مدیریت چرخه حیات توسط systemd، به ازای هر کانتینر یک فایل service. تولید خواهد شد. دستور rm برای حذف کانتینرها پیاده‌سازی می‌شود که به صورت پیش‌فرض اجازه حذف کانتینرهای در حال اجرا را ندارد (مگر با استفاده از پرچم -f). دستور ps نیز باید قابلیت نمایش لیست کانتینرهای در حال اجرا و با پرچم -a لیست تمام کانتینرها به همراه کد خروجی و زمان توقف را داشته باشد. در بخش دوم، ابزار zocker-proxy توسعه داده می‌شود که نقش یک واسطه را روی ماشین فیزیکی ایفا می‌کند و خود هیچ کانتینری را اجرا نمی‌کند. این ابزار باید با حداقل دو ماشین مجازی از طریق پروتکل VSOCK ارتباط برقرار کند. وظیفه اصلی این ابزار، توزیع کانتینرها روی ماشین‌های مجازی بر اساس منابع موجود است. بدین منظور، ابزار واسطه پرچم‌های -cpu و -memory را دریافت کرده و پیش از ارسال دستور اجرا، منابع اختصاص یافته به کانتینرهای قبلی را روی هر ماشین بررسی می‌کند تا کانتینر جدید را به ماشین آزادتر اختصاص دهد. در صورت نبود منابع کافی، اجرای کانتینر با خطا مواجه می‌شود. همچنین ابزار واسطه باید لیست کانتینرهای هر ماشین را نمایش دهد و تغییرات وضعیت آن‌ها را به‌روز نگه دارد. در بخش امتیازی، شبکه‌سازی پیشرفته با دستور connect zocker-proxy و IP رنج ۲۴/۰.۰.۰.۱۰ پیاده می‌شود. با لینک‌های veth و iptables، ارتباط بین کانتینرها حتی روی ماشین‌های متفاوت و ارسال Ping فراهم می‌گردد.

فهرست مطالب

۱	مقدمه	۱
۱	۱-۱ تعریف مسئله	۱
۲	۲-۱ اهداف پروژه	۲
۳	۳-۱ ابزارهای مورد استفاده	۳
۴	۲ مدیریت کانتینرهای محلی	۴
۴	۱-۲ پیاده سازی	۴
۴	۱-۱-۲ پیاده سازی استاندارد OCI	۴
۵	۲-۱-۲ مدیریت چرخه حیات با Systemd	۵
۵	۳-۱-۲ پیاده سازی دستورات اصلی	۵
۶	۲-۲ کدهای منبع:	۶
۶	۱-۲-۲ تحلیل منطق موتور اجرایی (core_engine.py):	۶
۶	۲-۲-۲ تحلیل منطق مدیریت چرخه حیات (manager.py):	۶
۷	۳-۲-۲ تحلیل منطق ایجنت شناسی (register.py):	۷
۸	۳ هسته اجرایی و مکانیزم‌های ایزولاسیون	۸
۸	۱-۳ ایزولاسیون با استفاده از فضاها (Namespaces)	۸
۸	۲-۳ مدیریت منابع سخت‌افزاری (Cgroups v2)	۸
۹	۱-۲-۳ ساختار و سلسله‌مراتب کنترلی	۹

۹	۲-۲-۳ مکانیزم الحاق پردازش (Attachment)
۹	۳-۲-۳ پاکسازی و آزادسازی منابع
۱۰	۳-۳ مجازی سازی ریشه فایل سیستم و Chroot
۱۰	۴-۳ مدیریت وضعیت
۱۱	۴ سیستم توزیع کننده Zocker-Proxy
۱۱	۱-۴ زیرساخت ارتباطی VSOCK :
۱۱	۲-۴ تخصیص منابع:
۱۲	۳-۴ فرآیند اجرای کانتنر از راه دور (Remote Execution)
۱۲	۴-۴ مکانیزم شناسایی خودکار (Node Discovery)
۱۳	۵-۴ کدهای منبع بخش توزیع شده:
۱۳	۱-۵-۴ تحلیل منطق ابزار واسط (zocker-proxy.py):
۱۴	۲-۵-۴ تحلیل منطق عامل اجرایی (zocker-agent.py):
۱۴	۳-۵-۴ تحلیل منطق مدیریت محلی (zocker.py):
۱۵	۵ زیرساخت شبکه و ارتباطات بین کانتنری (بخش امتیازی)
۱۵	۱-۵ ایزولاسیون شبکه در سطح هسته
۱۶	۲-۵ جریان اجرای دستور connect در معماری توزیع شده
۱۷	۳-۵ پیاده سازی مکانیزم اتصال و ایزولاسیون شبکه ای کانتنرها در manager.py
۱۷	۴-۵ تحلیل امنیتی طراحی
۱۸	۵-۵ نتایج تجربی
۱۸	۶-۵ جمع بندی
۱۹	۶ تست، ارزیابی و نتایج عملیاتی
۱۹	۱-۶ فاز اول: صحت سنجی موتور محلی (Local Engine)
۲۱	۲-۶ فاز دوم: بررسی سناریو توزیع کانتنر (Scheduling)

۳-۶ فاز سوم: صحت‌سنجی شبکه بین‌کانتینری (بخش امتیازی) ۲۵

۴-۶ جمع‌بندی نهایی ۲۶

۷ چالش‌ها و راه‌حل‌ها ۲۷

۱-۰-۷ چالش‌های مدیریت کانتینرها ۲۷

۲-۰-۷ چالش‌های Zocker Proxy ۲۷

۳-۰-۷ چالش ایزولاسیون کامل و مدیریت مسیرهای مطلق (Absolute Paths) . ۲۸

۴-۰-۷ تداخل مدیریت حافظه در محیط‌های مجازی (WSL2/VM) ۲۸

۵-۰-۷ ایزولاسیون فایل سیستم مجازی و مکانیزم نقاط مونت ۲۸

۶-۰-۷ هماهنگی سطح دسترسی در تعاملات بین‌گروهی ۲۹

پیوست: کدهای منبع پروژه ۳۰

فصل ۱

مقدمه

۱-۱ تعریف مسئله

در دنیای توسعه نرم افزار، نیاز به محیط‌های ایزوله برای اجرای برنامه‌ها بدون تداخل با سیستم میزبان بسیار مهم است. مسئله اصلی این پروژه، طراحی و ساخت یک Container Engine بومی است که بتواند بدون وابستگی به ابزارهای آماده‌ای مثل Docker، مستقیماً با هسته لینوکس تعامل داشته باشد. چالش‌های کلیدی که در این مسئله باید حل می‌شد عبارتند از:

- **ایزولاسیون منطقی:** چگونه می‌توان یک پردازنده را طوری محدود کرد که تصور کند تنها پردازنده در حال اجرا روی سیستم است و به فایل‌های حساس میزبان دسترسی نداشته باشد؟
- **کنترل سخت‌افزاری:** چگونه می‌توان برای هر کانتینر سقف مصرف رم و توان پردازشی مشخص کرد تا از اشباع منابع سیستم جلوگیری شود؟
- **مدیریت توزیع شده:** در مقیاس بزرگ، چگونه می‌توان کانتینرها را در چندین ماشین مجازی مختلف مدیریت کرد و منابع آزاد آنها را بررسی کرد؟

۲-۱ اهداف پروژه

این پروژه با هدف دستیابی به یک سامانه مدیریت کانتینر پایدار به نام Zocker تعریف شده است که اهداف زیر را دنبال می‌کند:

- **پیاده‌سازی هسته ایزوله (Core Engine):** ایجاد یک محیط Sandbox با استفاده از قابلیت‌های Namespaces لینوکس برای جدا کردن PID، شبکه و نام میزبان.
- **مدیریت هوشمند منابع (Resource Management):** بهره‌گیری از تکنولوژی Cgroups v2 برای اعمال محدودیت‌های سخت‌گیرانه روی حافظه و پردازنده.
- **انتزاع سیستم فایل (Filesystem Abstraction):** استفاده از مکانیزم‌های chroot و bind mount برای ایجاد یک ریشه (Root) اختصاصی و امن برای هر کانتینر.
- **توزیع‌شدگی و Orchestration:** طراحی یک ساختار Proxy-Agent که بتواند کانتینرها را به صورت هوشمند بر اساس فضای خالی رم بین ماشین‌های مختلف توزیع کند (Load Balancing).
- **مدیریت چرخه حیات (Lifecycle Management):** امکان شروع، توقف، حذف و پایش کانتینرها به صورت خودکار و از راه دور.

۳-۱ ابزارهای مورد استفاده

برای رسیدن به اهداف ذکر شده، از مجموعه‌ای از ابزارهای سطح پایین سیستم‌عامل و زبان‌های برنامه‌نویسی استفاده شده است:

- **زبان برنامه‌نویسی Python:** به عنوان زبان اصلی برای پیاده‌سازی منطق مدیریت کانتینر و شبکه.
- **Linux Namespaces:** ابزار اصلی برای ایزولاسیون؛ شامل CLONE_NEWPID برای جدا کردن پردازنده‌ها، CLONE_NEWUTS برای تغییر Hostname و CLONE_NEWNS برای مدیریت نقاط اتصال فایل سیستم.
- **Control Groups (Cgroups v2):** لایه‌ی مدیریتی در `/sys/fs/cgroup` برای کنترل مستقیم سخت‌افزار توسط هسته لینوکس.
- **کتابخانه ctypes:** جهت فراخوانی مستقیم توابع سیستمی زبان C (System Calls) مانند unshare و mount از داخل پایتون.
- **پروتکل VSOCK:** برای برقراری ارتباط امن و سریع بین ماشین مجازی (Agent) و سیستم میزبان (Proxy).
- **Systemd:** برای مدیریت کانتینرها به عنوان سرویس‌های پس‌زمینه سیستم‌عامل جهت تضمین پایداری و قابلیت شروع مجدد خودکار.
- **کتابخانه psutil:** جهت استخراج دقیق وضعیت لحظه‌ای سخت‌افزار (مثل رم آزاد) در هر گره.

فصل ۲

مدیریت کانتینرهای محلی

در این فصل به بررسی پیاده‌سازی زیرساخت‌های لازم برای مدیریت کانتینرها در سطح نودهای عامل (Agent) می‌پردازیم. تمرکز اصلی بر روی استانداردسازی فرآیندها و استفاده از ابزارهای بومی لینوکس برای پایداری سرویس‌ها است.

۱-۲ پیاده‌سازی

۱-۱-۲ پیاده‌سازی استاندارد OCI

برای انطباق با استانداردهای مدیریت کانتینر (OCI)، سیستم باید بتواند وضعیت هر کانتینر را به صورت مجزا و ساختاریافته ذخیره کند.

- ایجاد ساختار دایرکتوری: تمام کانتینرها در مسیر توافق شده `/HOME/.zocker/containers` و در پوشه‌هایی با نام اختصاصی `UUID` ذخیره می‌شوند.
- فایل `state.json`: بلافاصله پس از اجرای کانتینر، فایل به نام `state.json` ایجاد می‌گردد. این فایل شامل اطلاعات حیاتی از جمله موارد زیر است:

- `ID`: شناسه یکتای کانتینر.
- `Status`: وضعیت فعلی (مثلاً `running` یا `stopped`).
- `PID`: شناسه پروسه اصلی در سیستم میزبان برای رهگیری مستقیم.

– Bundle Path: مسیر فایل‌های کانیتینر.

این فایل به عنوان (Source of Truth) عمل می‌کند و ابزارهای مدیریتی دیگر با خواندن آن، وضعیت کانیتینر را تشخیص می‌دهند.

۲-۱-۲ مدیریت چرخه حیات با Systemd

یکی از چالش‌های اصلی پروژه، اجرای کانیتینرها به صورت مستقل از ترمینال کاربر (Daemonless) بود. برای حل این چالش، از سیستم مدیریت سرویس‌های لینوکس (Systemd) استفاده شد.

- **تولید خودکار Unit File:** به ازای هر کانیتینر، یک فایل با پسوند service تولید می‌شود. در این فایل، دستور اجرای موتور اصلی کانیتینر (core_engine.py) به همراه پارامترهای لازم تعریف می‌گردد.
- **پایداری (Persistence):** با ثبت کانیتینر به عنوان یک سرویس سیستم‌دی، حتی در صورت بسته شدن ترمینال یا خروج کاربر از سیستم، کانیتینر به فعالیت خود در پس‌زمینه ادامه می‌دهد.
- **مدیریت هوشمند:** سیستم‌دی مسئولیت نظارت بر سلامت پروسه را بر عهده دارد و امکاناتی نظیر شروع مجدد خودکار (Auto-restart) در صورت بروز خطا را فراهم می‌کند.

۳-۱-۲ پیاده‌سازی دستورات اصلی

پس از استقرار زیرساخت سرویس‌دهی، ابزارهای مدیریتی برای کنترل کانیتینرها به شرح زیر پیاده‌سازی شدند:

- **دستور stop:** این دستور با ارسال سیگنال به سیستم‌دی، واحد سرویس مربوط به UUID کانیتینر را متوقف می‌کند. این کار باعث خروج امن کانیتینر و آپدیت وضعیت در فایل state.json می‌شود.
- **دستور restart:** از این دستور برای بازنشانی کانیتینر استفاده می‌شود. سیستم‌دی پروسه قبلی را خاتمه داده و کانیتینر را با همان تنظیمات اولیه مجدداً بارگذاری می‌کند.
- **دستور ps:** این دستور با پیمایش مسیر کانیتینرها، تمام پوشه‌های موجود را بررسی کرده و با خواندن فایل state.json در هر پوشه، لیستی از کانیتینرهای فعال و غیرفعال را به همراه زمان اجرا و منابع مصرفی نمایش می‌دهد.

- دستور `rm`: برای حذف کانتینر، ابتدا وضعیت آن چک می‌شود. اگر کانتینر در حال اجرا باشد، سیستم اجازه حذف را نمی‌دهد (ایمنی سیستم). در صورت استفاده از پرچم `-f` یا متوقف بودن کانتینر، تمام فایل‌های سرویس و دایرکتوری‌های مربوطه پاکسازی می‌شوند.

۲-۲ کدهای منبع:

در این بخش به بررسی فنی و تحلیل الگوریتم‌های به‌کاررفته در سه ماژول اصلی سیستم پرداخته می‌شود. (کدهای این بخش در پیوست آمده است).

۱-۲-۲ تحلیل منطق موتور اجرایی (`core_engine.py`):

این فایل هسته عملیاتی کانتینر است که وظیفه تعامل مستقیم با هسته لینوکس جهت ایزولاسیون را بر عهده دارد:

- ایزولاسیون فضای نام (`Namespaces`): این کد با فراخوانی سیستمی `libc.unshare` و استفاده از پرچم‌های `CLONE_NEWPID` و `CLONE_NEWNS`، محیط پردازشی و فایل سیستم کانتینر را کاملاً از میزبان جدا می‌کند.

- مکانیزم پایداری سرویس: برای جلوگیری از خاتمه زودهنگام سرویس در `systemd`، از تابع `os.waitpid` در پروسه والد استفاده شده است. این منطق باعث می‌شود والد تا زمان پایان کار کانتینر (فرزند) باز بماند و وضعیت سرویس را `active` نگه دارد.

- مدیریت `Rootfs`: تابع `manage_rootfs` با استفاده از دستورات `--bind mount`، نقاط اتصال حیاتی میزبان را به صورت `Read-Only` در اختیار کانتینر قرار می‌دهد.

۲-۲-۲ تحلیل منطق مدیریت چرخه حیات (`manager.py`):

این ماژول به عنوان رابط مدیریتی بین کاربر و سیستم عامل عمل کرده و منطق `Daemonless` را پیاده‌سازی می‌کند:

- تولید دینامیک واحد سرویس: تابع `create_and_start_service` تنظیمات کانتینر را به یک فایل `service` استاندارد تبدیل کرده و در مسیر `/etc/systemd/system/` بارگذاری می‌کند.

- مدیریت هوشمند خروج: با استفاده از دستور ExecStopPost در فایل سرویس، تضمین می‌شود که حتی در صورت کرش کردن کانتنر، وضعیت در فایل state.json به صورت خودکار به stopped تغییر یابد.

- امنیت در عملیات حذف: تابع remove_container با چک کردن وضعیت لایو از طریق systemctl is-active، از حذف فیزیکی کانتنرهای در حال اجرا بدون پرچم اجباری جلوگیری می‌کند.

۳-۲-۲ تحلیل منطق ایجنت شناسی (register.py):

این کد وظیفه معرفی نود عامل به نود مرکزی (پراکسی) را بر عهده دارد:

- کشف منابع سخت‌افزاری: با بهره‌گیری از کتابخانه psutil، ظرفیت واقعی RAM و تعداد هسته‌های CPU استخراج شده و برای درک بهتر سیستم توزیع‌شده، به واحدهای استاندارد (گیگابایت) تبدیل می‌شوند.

- ارتباط شبکه مبتنی بر سوکت: منطق ارتباطی بر پایه پروتکل TCP/IP و سوکت‌های AF_INET استوار است که ارسال ایمن و تضمین‌شده داده‌های اینونتوری به IP مقصد (192.168.1.104) را بر عهده دارد.

فصل ۳

هسته اجرایی و مکانیزم‌های ایزولاسیون

در این فصل، لایه زیرین کانتینر که وظیفه تعامل مستقیم با هسته لینوکس، ایزولاسیون منابع و مدیریت سخت‌افزار را بر عهده دارد، بررسی می‌شود.

۳-۱ ایزولاسیون با استفاده از فضاهای نام (Namespaces)

برای ایجاد یک محیط کاملاً مستقل، از فراخوان سیستمی `unshare` استفاده شده است. این بخش که در ماژول `isolation.py` پیاده‌سازی شده، ایزولاسیون را در سطوح زیر تضمین می‌کند:

- **PID Namespace**: با استفاده از پرچم `CLONE_NEWPID`، پردازش کانتینر از جدول پردازش‌های سیستم میزبان جدا شده و کانتینر خود را به عنوان پردازش شماره ۱ شناسایی می‌کند.
- **UTS Namespace**: جهت اختصاص نام میزبان (`Hostname`) مجزا برای هر کانتینر که توسط تابع `set_container_hostname` انجام می‌گیرد.
- **Mount Namespace**: برای ایجاد نقاط اتصال (`Mount points`) اختصاصی و جدا کردن سیستم فایل کانتینر.

۳-۲ مدیریت منابع سخت‌افزاری (Cgroups v2)

توانایی محدودسازی منابع فیزیکی مصرفی از ویژگی‌های اصلی تکنولوژی کانتینر است. در این پروژه، این وظیفه بر عهده ماژول `resources.py` است که از قابلیت `Control Groups v2` استفاده می‌کند.

۱-۲-۳ ساختار و سلسله مراتب کنترلی

برخلاف نسخه قدیمی، در Cgroups v2 تمامی منابع تحت یک سلسله مراتب واحد مدیریت می شوند. کلاس ResourceManager با ایجاد یک دایرکتوری اختصاصی در مسیر `/sys/fs/cgroup`، یک قلمرو کنترلی جدید برای هر کانتینر ایجاد می کند. فرآیند اعمال محدودیت در سه لایه عملیاتی صورت می پذیرد:

- **فعال سازی کنترلرها:** ابتدا با نوشتن در فایل `cgroup.subtree_control`، زیرسیستم های حافظه (memory)، پردازنده (cpu) و پردازنده ها (pids) برای زیرمجموعه های جدید فعال می گردند.

- **محدود سازی حافظه (RAM):** مقدار حد آستانه از فایل پیکربندی استخراج شده و در فایل `memory.max` نوشته می شود. این مقدار به هسته لینوکس دستور می دهد که در صورت فراتر رفتن مصرف کانتینر از این مرز، مکانیزم OOM Killer را فعال کرده یا عملیات تخصیص حافظه را متوقف کند.

- **زمان بندی پردازنده (CPU):** با استفاده از فایل `cpu.max`، سهم کانتینر از بازه های زمانی پردازش (Quotas) تعیین می شود. این مقدار به صورت جفت عدد (سقف استفاده در بازه زمانی مشخص) اعمال می گردد تا از اشغال کامل پردازنده توسط یک کانتینر مخرب جلوگیری شود.

۲-۲-۳ مکانیزم الحاق پردازنده (Attachment)

حیاتی ترین مرحله در مدیریت منابع، متصل کردن پردازنده در حال اجرا به گروه کنترل ساخته شده است. در متد `attach`، شناسه پردازنده (PID) فرزند که توسط `os.fork` ایجاد شده است، در فایل `cgroup.procs` درج می گردد. به محض این درج، تمام محدودیت های تعریف شده در لایه های قبلی به صورت آنی توسط زمان بند هسته لینوکس بر روی آن پردازنده و تمام فرزندان احتمالی آن اعمال می شود.

۳-۲-۳ پاک سازی و آزاد سازی منابع

پس از اتمام چرخه حیات کانتینر و خروج فرآیند، سیستم موظف به آزاد سازی منابع است. متد `remove` در کلاس مدیریت منابع، وظیفه حذف دایرکتوری های ساخته شده در فایل سیستم مجازی `cgroup` را بر عهده دارد تا از انباشت گروه های خالی و اتلاف حافظه سیستم میزبان جلوگیری شود.

۳-۳ مجازی سازی ریشه فایل سیستم و Chroot

پیش از اجرای نهایی دستور کانتینر، ریشه فایل سیستم با استفاده از مکانیزم Bind Mount آماده می شود. دایرکتوری های حیاتی مانند `/bin` و `/lib` به صورت Read-Only به داخل ریشه جدید منتقل شده و سپس با دستور `os.chroot`، محیط اجرای فرآیند کاملاً محدود می گردد.

۴-۳ مدیریت وضعیت

برای هماهنگی با لایه های بالاتر (پراکسی و ایجنت)، ماژول هسته موظف است وضعیت لحظه ای کانتینر را در فایل `state.json` به روزرسانی کند. این اطلاعات شامل زمان شروع، وضعیت فعلی (Running/Stopped) و کد خروجی فرآیند پس از اتمام کار می باشد.

فصل ۴

سیستم توزیع کننده Zocker-Proxy

در این بخش، جزئیات عملیاتی پیاده‌سازی ابزار واسط (Proxy) و عامل‌های مستقر در ماشین‌های مجازی (Agents) شرح داده می‌شود. پیاده‌سازی در چهار فاز اصلی انجام شده است.

۴-۱ زیرساخت ارتباطی VSOCK :

- در سمت Agent: یک سوکت با آدرس AF_VSOCK طراحی شد که روی پورت 9999 در حالت Listen قرار دارد تا دستورات اجرایی را دریافت کند.
- در سمت Proxy: یک سیستم Threading ایجاد شد تا همزمان هم دستورات کاربر را دریافت کند و هم روی پورت 8888 منتظر پیام‌های ثبت‌نام (Registration) از VM‌های جدید باشد.

۴-۲ تخصیص منابع:

- مدیریت وضعیت (Stateful Tracking): در هر VM، فایلی به نام state.json در مسیر ~/.zocker/containers/ ایجاد شد. این فایل لیست UUID کانتینرهای در حال اجرا و منابع تخصیص یافته را نگه می‌دارد.
- محاسبه فضای آزاد: قبل از اجرا، پراکسی یک درخواست get_stats به تمام ایجنت‌ها می‌فرستد. ایجنت با خواندن فایل وضعیت، مجموع منابع رزرو شده را از کل ظرفیت ماشین کسر کرده و عدد نهایی را برمی‌گرداند.

- انتخاب هوشمند: پراکسی با مقایسه اعداد دریافتی، کانتینر را به ماشینی می‌فرستد که بیشترین ظرفیت خالی را دارد (Load Balancing). در صورت عدم کفایت منابع در تمام نودها، فرآیند با چاپ خطای عملیاتی متوقف می‌شود.

۳-۴ فرآیند اجرای کانتینر از راه دور (Remote Execution)

پس از انتخاب مقصد، مراحل عملیاتی زیر طی می‌شود:

- تولید UUID: پراکسی یک شناسه یکتا برای کانتینر تولید می‌کند.
- ارسال پکیج: تنظیمات JSON کانتینر به همراه محدودیت‌های منابع (Memory Limit و CPU Shares) به VM مقصد ارسال می‌شود.
- فراخوانی Core: ایجنت در مقصد، اطلاعات را ذخیره کرده و با استفاده از ماژول subprocess پایتون، کد اصلی پروژه را به صورت Background اجرا می‌کند.

۴-۴ مکانیزم شناسایی خودکار (Node Discovery)

- برای حذف نیاز به وارد کردن دستی CIDها، فرآیند ثبت‌نام خودکار به ترتیب زیر پیاده‌سازی شد:
- به محض اجرای zocker-agent.py در دیبیا، اسکریپت با استفاده از کتابخانه psutil ظرفیت کل RAM و CPU ماشین را استخراج می‌کند.
 - ایجنت یک بسته JSON حاوی این اطلاعات را به CID شماره ۲ (آدرس ثابت Host در VSOCK) ارسال می‌کند.
 - پراکسی پس از دریافت، این اطلاعات را در یک Dictionary به همراه زمان آخرین اتصال ذخیره می‌کند.

۴-۵ کدهای منبع بخش توزیع شده:

در ادامه، کدهای اصلی پیاده‌سازی شده برای این بخش که در بخش پیوست کدها قابل مشاهده است، آورده شده است.

۴-۵-۱ تحلیل منطق ابزار واسط (zocker-proxy.py):

این اسکریپت به عنوان واحد کنترل مرکزی (Control Plane) عمل می‌کند و وظیفه هماهنگی بین گره‌های مختلف را بر عهده دارد.

– مدیریت همزمانی: با استفاده از کتابخانه `threading`، این ابزار می‌تواند همزمان با مدیریت واسط کاربری (CLI)، به پیام‌های ثبت‌نام ماشین‌های مجازی جدید نیز گوش دهد. استفاده از `Lock` در دسترسی به دیکشنری `nodes`، از بروز تداخل داده‌ای (Race Condition) جلوگیری می‌کند.

– الگوریتم توزیع بار `Scheduling`: تابع `select_node` پیاده‌سازی یک الگوریتم هوشمند بر پایه کمترین منابع تخصیص یافته است. این تابع با پیمایش لیست گره‌ها، گره‌ای را انتخاب می‌کند که بیشترین فضای خالی `Free RAM` را داشته باشد.

– تجرید دستورات `Abstraction`: پراکسی دستورات ساده کاربر مانند `run` یا `ps` را به بسته‌های `JSON` استاندارد تبدیل کرده و از طریق سوکت‌های `VSOCK` به ایجنت مقصد هدایت می‌کند.

۴-۵-۲ تحلیل منطق عامل اجرایی (zocker-agent.py):

این اسکریپت در سطح Worker Nodes اجرا شده و مسئولیت تعامل مستقیم با سیستم عامل دبیان را دارد.

– پایداری از طریق سرویس دهی: ایجنت از ماژول manager برای تبدیل کانینر به یک Sys-temd Service استفاده می کند. این کار باعث می شود کانینرها در صورت کرش کردن، توسط سیستم عامل بازیابی شوند.

– مانیتورینگ سخت افزاری: ایجنت با استفاده از کتابخانه psutil، ظرفیت واقعی سیستم را استخراج کرده و با ترکیب آن با داده های موجود در state.json، یک گزارش از وضعیت منابع به پراکسی ارائه می دهد.

– ایزولاسیون دستورات: هر دستور دریافت شده از پراکسی، در یک دایرکتوری مجزا بر اساس UUID کانینر ایزوله می شود تا تداخلی بین فایل های پیکربندی کانینرهای مختلف رخ ندهد.

۴-۵-۳ تحلیل منطق مدیریت محلی (zocker.py):

این ابزار به عنوان لایه واسط بین کاربر و هسته مدیریت کانینر عمل کرده و منطق چرخه حیات (Lifecycle) را پیاده سازی می کند.

– مدیریت هوشمند پیکربندی: در بخش دستور run، اگر کانینری با همان نام از قبل وجود داشته باشد، ابتدا تفاوت فایل config.json جدید را با نسخه قبلی چک می کند. در صورت وجود تغییر، کانینر قدیمی را به صورت خودکار حذف (Cleanup) و نسخه جدید را جایگزین می کند.

– یکپارچگی با بخش توزیع شده: این فایل برای مدیریت محلی است، اما ساختار ذخیره سازی وضعیت (State) در آن کاملاً با ایجنت توزیع شده هماهنگ است، به طوری که ایجنت می تواند از همین منطق برای مدیریت کانینرهای راه دور استفاده کند.

فصل ۵

زیرساخت شبکه و ارتباطات بین کانتینری (بخش امتیازی)

در این فصل، معماری شبکه در پلتفرم Zocker از منظر طراحی سیستم عامل و هماهنگی اجزای یک سیستم توزیع شده تحلیل می شود. این زیرساخت به صورت کامل بر قابلیت های بومی هسته لینوکس تکیه دارد و بدون استفاده از موتورهای کانتینری آماده پیاده سازی شده است. پیاده سازی شبکه در فایل های `zocker-proxy.py`، `zocker-agent.py`، `manager.py`، `core_engine.py` و `isolation.py` توزیع شده است.

هدف این معماری، ایجاد ارتباط ایزوله، کم تأخیر و کنترل پذیر بین کانتینرها در سطح لایه ۲ شبکه است.

۵-۱ ایزولاسیون شبکه در سطح هسته

ایزولاسیون شبکه از لحظه ایجاد کانتینر در فایل `core_engine.py` آغاز می شود. در تابع راه اندازی کانتینر، سیستم کال `unshare` با فعال سازی پرچم های `Namespace` فراخوانی می شود. در کنار `CLONE_NEWPID` و `CLONE_NEWNS`، ایزولاسیون شبکه از طریق `CLONE_NEWNET` که در ماژول `isolation.py` تعریف شده اعمال می گردد. تابع `apply_isolation` در این فایل مسئول ایجاد های `Namespace` جدید شامل:

— Network Namespace

Mount Namespace —

UTS Namespace —

IPC Namespace —

می‌باشد.

در نتیجه:

- هر کانتینر جدول مسیریابی مستقل دارد.
- هیچ دسترسی پیش فرضی به اینترفیس‌های میزبان وجود ندارد.
- تداخل IP بین کانتینرها رخ نمی‌دهد.

۲-۵ جریان اجرای دستور connect در معماری توزیع شده

اتصال دو کانتینر از طریق همکاری سه لایه انجام می‌شود:

۱. لایه پروکسی — فایل `zocker-proxy.py`:

در این فایل، متد `send_command` مسئول ارسال پیام کنترلی به ماشین مجازی مقصد است. هنگام اجرای دستور `connect` در CLI، یک پیام JSON شامل شناسه دو کانتینر تولید شده و از طریق سوکت `AF_VSOCK` به پورت 9999 ارسال می‌شود.

۲. لایه عامل — فایل `zocker-agent.py`:

در این فایل، تابع `listen` روی پورت فرمان در حالت انتظار قرار دارد. پس از دریافت پیام، مقدار فیلد `cmd` بررسی شده و در صورت برابر بودن با `connect`، درخواست به تابع `connect_containers` در ماژول مدیریت ارجاع داده می‌شود.

۳. لایه مدیریت — فایل `manager.py`:

در این فایل، تابع `connect_containers` عملیات واقعی پیکربندی شبکه را انجام می‌دهد. ابتدا با مراجعه به فایل `state.json` هر کانتینر، PID واقعی آن استخراج می‌شود. این PID برای ورود به `Network Namespace` کانتینر مورد استفاده قرار می‌گیرد.

این تفکیک لایه‌ها باعث جداسازی کامل `Control Plane` از `Data Plane` شده و معماری سیستم را ماژولار و توسعه‌پذیر می‌کند.

۳-۵ پیاده‌سازی مکانیزم اتصال و ایزولاسیون شبکه‌ای کانتینرها

در `manager.py`

تابع `connect_containers` در فایل `manager.py` مسئول ایجاد ساختار فیزیکی شبکه مجازی است. مراحل انجام‌شده در این تابع به صورت زیر است:

– ایجاد Bridge:

یک اینترفیس از نوع `bridge` با نام `zocker-br0` ایجاد می‌شود. این Bridge نقش یک سوئیچ لایه ۲ را ایفا می‌کند. سپس آدرس `10.0.0.1/24` به آن اختصاص داده می‌شود.

– ایجاد `Pair Veth` برای هر کانتینر:

برای هر کانتینر یک جفت اینترفیس `veth` ساخته می‌شود. یک سر این جفت به Bridge متصل می‌شود و سر دیگر به `Namespace` کانتینر منتقل می‌گردد.

– تزریق به `Namespace` با استفاده از `nsenter`:

پس از انتقال رابط به `Namespace` کانتینر، از ابزار `nsenter` برای ورود به فضای نام شبکه استفاده شده و اینترفیس به `eth0` تغییر نام داده می‌شود. سپس IP های `10.0.0.2` و `10.0.0.3` به کانتینرها اختصاص داده شده و اینترفیس فعال می‌شود.

این طراحی باعث می‌شود انتقال بسته‌ها مستقیماً در سطح هسته لینوکس انجام گیرد و هیچ NAT یا لایه مجازی‌سازی اضافه‌ای در مسیر داده وجود نداشته باشد.

۴-۵ تحلیل امنیتی طراحی

امنیت شبکه در Zocker مبتنی بر سه اصل است:

– ایزولاسیون کامل با استفاده از `Namespaces` (در `isolation.py`)

– کنترل متمرکز دستورات شبکه در `manager.py`

– استفاده از `VSOCK` برای ارتباط کنترل بدون وابستگی به شبکه داخلی

از آنجا که هیچ مسیری به شبکه میزبان تنظیم نشده است، کانتینرها تنها در صورت تعریف صریح Bridge قادر به برقراری ارتباط خواهند بود.

۵-۵ نتایج تجربی

برای ارزیابی عملکرد شبکه، تست ping بین آدرس‌های 10.0.0.2 و 10.0.0.3 انجام شد. نتایج نشان داد:

- تبادل موفق بسته‌های ICMP

- تأخیر کمتر از 0.1 میلی‌ثانیه

- عدم دسترسی به شبکه میزبان

این نتایج نشان می‌دهد پیاده‌سازی انجام‌شده در فایل `manager.py` به‌درستی `Bridge` و `Veth` را در سطح هسته پیکربندی کرده است.

۵-۶ جمع‌بندی

پیاده‌سازی شبکه در Zocker نشان می‌دهد که با استفاده از قابلیت‌های بومی لینوکس و بدون وابستگی به موتورهای آماده، می‌توان یک زیرساخت شبکه ایزوله و کارآمد ایجاد کرد. همکاری ماژول‌های `zocker-agent.py`، `zocker-proxy.py`، `manager.py`، `core_engine.py` و `isolation.py` معماری منسجم و ماژولاری را شکل داده است که هم از نظر امنیت و هم از نظر کارایی در سطح مطلوبی قرار دارد.

فصل ۶

تست، ارزیابی و نتایج عملیاتی

در این فصل، صحت عملکرد زیرسیستم‌های پلتفرم Zocker در دو فاز عملیاتی مجزا مورد آزمون قرار می‌گیرد. هدف از این تست‌ها، اطمینان از صحت ایزولاسیون در سطح محلی و هماهنگی اجزا در محیط توزیع شده تحت کنترل پروکسی است.

۶-۱ فاز اول: صحت‌سنجی موتور محلی (Local Engine)

در این فاز، توانایی موتور اصلی در ایجاد محیط ایزوله در سطح یک نود واحد، بدون دخالت لایه توزیع‌کننده و با استفاده از ابزار `zocker.py` مورد ارزیابی قرار گرفت.

– تست ایزولاسیون فرآیندها (PID Isolation): با اجرای دستور `ps aux` در داخل کانتینر، مشاهده گردید که جدول فرآیندها کاملاً از سیستم میزبان تفکیک شده است. این خروجی نشان می‌دهد که پرچم `CLONE_NEWPID` در فایل `core_engine.py` به درستی عمل کرده و کانتینر تنها فرآیندهای داخلی خود را مشاهده می‌کند.

– تست ایزولاسیون فایل سیستم: با اجرای دستور `ls` در محیط کانتینر، اطمینان حاصل شد که کانتینر تنها به دایرکتوری `rootfs` اختصاصی خود دسترسی دارد و فایل سیستم میزبان کاملاً از دسترس آن خارج شده است.


```

navid@navid:~/uni/T5/OS/Project/zocker_project/src$ sudo python3 zocker.py run config.json
[sudo] password for navid:
[+] Container 8cd50bb6 started.
navid@navid:~/uni/T5/OS/Project/zocker_project/src$ sudo python3 zocker.py exec 8cd50bb6 /bin/bash
[*] Entering container 8cd50bb6...
root@zocker-8cd50b:/# ls
bin dev etc lib lib64 proc sys usr
root@zocker-8cd50b:/# ls /
bin dev etc lib lib64 proc sys usr
root@zocker-8cd50b:/# ps aux
USER          PID %CPU %MEM    VSZ   RSS TTY      STAT START   TIME COMMAND
root           1  0.0  0.0   2808  1844 ?        S    00:17   0:00 sh -c tail -f /dev/null
root          10  0.0  0.0   8324  1988 ?        S    00:17   0:00 tail -f /dev/null
root          11  0.0  0.0  10212  4444 ?        S    00:19   0:00 /bin/bash
root          15  0.0  0.0  13492  4504 ?        R+   00:19   0:00 ps aux
root@zocker-8cd50b:/# hostname
zocker-8cd50b
root@zocker-8cd50b:/# exit
exit
navid@navid:~/uni/T5/OS/Project/zocker_project/src$ systemctl status zocker-8cd50bb6
● zocker-8cd50bb6.service - Zocker Container 8cd50bb6
   Loaded: loaded (/etc/systemd/system/zocker-8cd50bb6.service; linked; preset: enabled)
   Active: active (running) since Mon 2026-02-16 00:17:49 +0330; 5min ago
     Main PID: 314447 (python3)
        Tasks: 3 (limit: 18038)
      Memory: 6.1M (peak: 9.7M)
         CPU: 105ms
    CGroup: /system.slice/zocker-8cd50bb6.service
            └─314447 /usr/bin/python3 /home/navid/uni/T5/OS/Project/zocker_project/src/core_engine.py
              └─314481 sh -c "tail -f /dev/null"
                └─314490 tail -f /dev/null

Feb 16 00:17:49 navid systemd[1]: Started zocker-8cd50bb6.service - Zocker Container 8cd50bb6.
Feb 16 00:17:49 navid python3[314483]: mount: /home/navid/.zocker/containers/8cd50bb6/rootfs/proc: spec
Feb 16 00:17:49 navid python3[314483]: dmesg(1) may have more information after failed mount sys

```

شکل ۶-۱: خروجی دستور ps aux و اثبات ایزولاسیون فرایندها در سطح محلی و Daemonless

```

navid@navid:~/uni/T5/OS/Project/zocker_project/src$ sudo python3 zocker.py ps -a
CONTAINER ID   STATUS     PID       EXIT      AGE / FINISHED
-----
89487f45       running    11398     -         7h
008ace82       stopped    0         0         Exited (7h ago)
58d6c153       running    138395    -         4h
8e563782       running    1434101   -         7h
3f6a6984       running    136317    -         4h
719f53d4       running    93564     -         5h
d284ee93       running    137103    -         4h
92912ea8       stopped    0         0         Exited (8h ago)
container_B    stopped    0         -         unknown
c37ab5db       running    1406738   -         8h
9aa71d88       running    134624    -         4h
c5895a65       running    1422252   -         8h
a0871b37       running    126009    -         4h
344dde10       stopped    0         0         Exited (8h ago)
cd66a42e       running    133884    -         4h
b197758c       running    135606    -         4h
e64bb81c       running    1420218   -         8h
8cd50bb6       running    314481    -         6m
dd63b5c5       running    41019     -         6h
35539bea       running    1425200    -         7h
a397b7e0       stopped    0         0         Exited (8h ago)

```

شکل ۶-۲: خروجی دستور ps -a

```

navid@navid:~/uni/T5/OS/Project/zocker_project/src$ sudo python3 zocker.py ps -a
CONTAINER ID   STATUS     PID       EXIT      AGE / FINISHED
-----
89487f45       running    11398      -         2h
008ace82       stopped    0          0         Exited (2h ago)
8e563782       running    1434101    -         3h
719f53d4       running    93564      -         44m
92912ea8       stopped    0          0         Exited (3h ago)
container_B    stopped    0          -         unknown
c37ab5db       running    1406738    -         3h
c5895a65       running    1422252    -         3h
a0871b37       running    126009     -         14s
344dde10       stopped    0          0         Exited (3h ago)
e64bb81c       running    1420218    -         3h
dd63b5c5       running    41019      -         2h
35539bea       running    1425200    -         3h
a397b7e0       stopped    0          0         Exited (3h ago)

navid@navid:~/uni/T5/OS/Project/zocker_project/src$ cat /sys/fs/cgroup/zocker_a0871b37/memory.max
536870912

navid@navid:~/uni/T5/OS/Project/zocker_project/src$ cat /sys/fs/cgroup/zocker_a0871b37/cpu.max
1024 100000

```

شکل ۶-۳: بررسی Cgroups

```

navid@navid:~/uni/T5/OS/Project/zocker_project/src$ sudo python3 zocker.py run config.json
[+] Container a0871b37 started.

navid@navid:~/uni/T5/OS/Project/zocker_project/src$ sudo python3 zocker.py run --name a0871b37 config.js
on
[-] Container 'a0871b37' is already running with this exact config.

navid@navid:~/uni/T5/OS/Project/zocker_project/src$ sudo python3 zocker.py run --name a0871b37 config.json
[*] Config change detected for 'a0871b37'. Restarting...
[*] Stopping service zocker-a0871b37...
[*] Unmounting rootfs for a0871b37...
[+] Container a0871b37 started.

navid@navid:~/uni/T5/OS/Project/zocker_project/src$ sudo python3 zocker.py ps -a
CONTAINER ID   STATUS     PID       EXIT      AGE / FINISHED
-----
89487f45       running    11398      -         2h
008ace82       stopped    0          0         Exited (2h ago)
8e563782       running    1434101    -         3h
719f53d4       running    93564      -         44m
92912ea8       stopped    0          0         Exited (3h ago)
container_B    stopped    0          -         unknown
c37ab5db       running    1406738    -         3h
c5895a65       running    1422252    -         3h
a0871b37       running    126009     -         14s
344dde10       stopped    0          0         Exited (3h ago)
e64bb81c       running    1420218    -         3h
dd63b5c5       running    41019      -         2h
35539bea       running    1425200    -         3h
a397b7e0       stopped    0          0         Exited (3h ago)

```

شکل ۶-۴: تغییر دادن فایل config.json و اجرای دوباره ی کانتینر

۲-۶ فاز دوم: بررسی سناریو توزیع کانتینر (Scheduling)

در این بخش، تعامل عملیاتی بین zocker-proxy.py و zocker-agent.py مورد آزمون قرار گرفت تا صحت الگوریتم انتخاب هوشمند گره تایید شود.

– مکانیزم شناسایی (Node Discovery): به محض اجرای ایجنت در ماشین‌های مجازی، مشخصات سخت‌افزاری شامل حافظه و پردازنده استخراج شده و از طریق پروتکل VSOCK به پروکسی ارسال گردید. پروکسی این اطلاعات را در یک دیکشنری امن (nodes) ذخیره کرد.

– تخصیص هوشمند منابع: با صدور دستور run از سمت پروکسی، تابع select_node با مقایسه مقادیر alloc_mem در گره‌های مختلف، کانتینر را به نودی هدایت کرد که بیشترین فضای خالی را داشت. این فرآیند مانع از تجمع بار روی یک نود واحد می‌گردد.

```

navid@navid:~/uni/T5/OS/Project/zocker_project/src$ sudo python3 zocker-proxy.py
--- Zocker Proxy CLI (Final Version) ---
Commands: nodes, run, ps, stop, rm, exec, exit
zocker-proxy > [*] Discovery service started on port 8888...

[+] Node Registered/Updated: CID 4

[+] Node Registered/Updated: CID 3
nodes
CID          Total RAM (MB)  Allocated RAM (MB)  Free RAM (MB)
-----
4            14855           0                   14855
3            14635           0                   14635
zocker-proxy > run --memory 512 config.json
[*] Dispatching container 0ebede81 to VM 4...
Result: OK: Service Started
zocker-proxy > ps
VM   ID          STATUS    PID    AGE      EXIT
-----
4    0ebede81    running  4980   37s     -
4    673a78f4    running  4555   9m 20s  -
3    78787185    running  3567   18m 36s  -
zocker-proxy > nodes
CID          Total RAM (MB)  Allocated RAM (MB)  Free RAM (MB)
-----
4            14855           512                 14343
3            14635           0                   14635
zocker-proxy > run --memory 512
usage: zocker-proxy.py [-h] [--memory MEMORY] config
zocker-proxy.py: error: the following arguments are required: config
zocker-proxy > run --memory 512 config.json
[*] Dispatching container 091e4652 to VM 3...
Result: OK: Service Started
zocker-proxy > nodes
CID          Total RAM (MB)  Allocated RAM (MB)  Free RAM (MB)
-----
4            14855           512                 14343
3            14635           512                 14123

```

شکل ۵-۶: وضعیت گره‌ها و کانتینرهای فعال در همه‌ی vm ها در پروکسی و تخصیص منابع بر اساس ظرفیت باقی‌مانده ماشین‌های مجازی

```

navid@navid:~/uni/T5/OS/Project/zocker_project/src$ sudo python3 zocker-proxy.py
run --memory 512 config.json
[*] Dispatching container e3f1772c to VM 4...
Result: OK: Service Started
zocker-proxy > exec 4 e3f1772c hostname
--- Output from e3f1772c ---
zocker-e3f177
zocker-proxy > run --memory 20000 config.json
Error: No node has enough free RAM for this request.
zocker-proxy > stop 4 e3f1772c
OK: Container Stopped
zocker-proxy > ps

```

VM	ID	STATUS	PID	AGE	EXIT
4	953f4c7f	running	5330	9m 15s	-
4	0ebede81	running	4980	19m 21s	-
4	673a78f4	running	4555	28m 5s	-
4	a714cd43	running	5608	6m 26s	-
4	e3f1772c	stopped	0	1m 38s	0
3	091e4652	running	4146	17m 29s	-
3	78787185	running	3567	37m 21s	-

```

zocker-proxy > ps -a

```

VM	ID	STATUS	PID	AGE	EXIT
4	953f4c7f	running	5330	9m 18s	-
4	0ebede81	running	4980	19m 24s	-
4	673a78f4	running	4555	28m 8s	-
4	a714cd43	running	5608	6m 29s	-
4	e3f1772c	stopped	0	1m 40s	0
3	091e4652	running	4146	17m 32s	-
3	78787185	running	3567	37m 23s	-

شکل ۶-۶: تست کردن دستور stop و فلگ memory در پروکسی

```

zocker-proxy > ps -a

```

VM	ID	STATUS	PID	AGE	EXIT
4	953f4c7f	running	5330	9m 28s	-
4	0ebede81	running	4980	19m 34s	-
4	673a78f4	running	4555	28m 17s	-
4	a714cd43	running	5608	6m 38s	-
4	e3f1772c	stopped	0	1m 50s	0
3	091e4652	running	4146	17m 41s	-
3	78787185	running	3567	37m 33s	-

```

zocker-proxy > rm 4 a714cd43
Result from VM 4: (False, 'Error: Container is running. Use -f to force remove.')
zocker-proxy > rm -f 4 a714cd43
CLI Error: invalid literal for int() with base 10: '-f'
zocker-proxy > rm 4 714cd43 -f
Result from VM 4: None
zocker-proxy > rm 4 a714cd43
Result from VM 4: (False, 'Error: Container is running. Use -f to force remove.')
zocker-proxy > rm 4 a714cd43 -f
Result from VM 4: (True, 'Container removed successfully.')
zocker-proxy > ps

```

VM	ID	STATUS	PID	AGE	EXIT
4	953f4c7f	running	5330	11m 59s	-
4	0ebede81	running	4980	22m 5s	-
4	673a78f4	running	4555	30m 48s	-
4	e3f1772c	stopped	0	4m 21s	0
3	091e4652	running	4146	20m 12s	-
3	78787185	running	3567	40m 4s	-

شکل ۶-۷: تست کردن دستور rm و گرفتن خطا و در ادامه تست کردن با فلگ f

```

zocker-proxy > run --memory 256 config.json
[*] Dispatching container fe735349 to VM 3...
Result: OK: Service Started
zocker-proxy > nodes
CID      Total RAM (MB)  Allocated RAM (MB)  Free RAM (MB)
-----
4        14855           512                 14343
3        14635           256                 14379
zocker-proxy > ps
VM  ID      STATUS  PID  AGE  EXIT
---
4   953f4c7f  running 5330  37m 0s  -
4   0ebede81  running 4980  47m 6s  -
4   673a78f4  running 4555  55m 49s  -
4   e3f1772c  stopped 0     29m 22s  0
3   fe735349  running 4616  10m 33s  -
3   091e4652  running 4146  45m 13s  -
3   78787185  running 3567  65m 5s  -

```

atashinbar-alavi on QEMU/KVM

Feb 15 4:46 PM

```

Terminal
KeyboardInterrupt
atashinbar-alavi@atashinbar-alavi:~$ sudo python3 zocker-agent.py
--- Zocker Agent Starting ---
[*] Trying to register at Proxy (Host CID 2)...
[*] Agent is listening for commands on VSOCK port 9999...
[*] Successfully registered at Proxy!
^CTraceback (most recent call last):
  File "/home/atashinbar-alavi/zocker-agent.py", line 127, in <module>
    time.sleep(1)
KeyboardInterrupt

atashinbar-alavi@atashinbar-alavi:~$ sudo python3 zocker-agent.py
--- Zocker Agent Starting ---
[*] Trying to register at Proxy (Host CID 2)...
[*] Agent is listening for commands on VSOCK port 9999...
[*] Successfully registered at Proxy!
[#] Received command: 'ps' from Host (CID 2)
[#] Received command: 'ps' from Host (CID 2)
[#] Received command: 'ps' from Host (CID 2)
[#] Received command: 'ps' from Host (CID 2)
[#] Received command: 'ps' from Host (CID 2)
[#] Received command: 'run' from Host (CID 2)
[*] Starting container creation: fe735349
[+] Container fe735349 service started successfully.
[#] Received command: 'ps' from Host (CID 2)
[#] Received command: 'run' from Host (CID 2)
[*] Starting container creation: ac8c6c40
[+] Container ac8c6c40 service started successfully.
^CTraceback (most recent call last):
  File "/home/atashinbar-alavi/zocker-agent.py", line 127, in <module>
    time.sleep(1)
KeyboardInterrupt

atashinbar-alavi@atashinbar-alavi:~$

```

```

Terminal
atashinbar-alavi@atashinbar-alavi:~$ sudo nsenter -t 4616 -m -u -i -n -p /bin/bash
[sudo] password for atashinbar-alavi:
root@zocker-fe7353:/# ls
bin  etc          initrd.img.old  lost+found  opt    run    sys    var
boot home      lib             media       proc    sbin  tmp    vmlinuz
dev  initrd.img  lib64          mnt         root    srv   usr    vmlinuz.old
root@zocker-fe7353:/# exit
exit

```

شکل ۶-۸: تست پایداری سرویس و دسترسی به کانتینر از طریق PID مربوطه که در پروکسی بدست آمده

۳-۶ فاز سوم: صحت‌سنجی شبکه بین کانتینری (بخش امتیازی)

این آزمون نهایی، هماهنگی کامل ماژول‌های `manager.py` و `zocker-agent.py` را برای ایجاد زیرساخت شبکه لایه ۲ نشان می‌دهد.

– **اتصال و آدرس‌دهی:** پس از ایجاد دو کانتینر بر روی یک گره، دستور `connect` صادر گردید. ایجنت با استخراج PIDها از فایل `state.json`، رابط‌های `veth` را ساخته و به فضای نام کانتینرها تزریق کرد.

– **تست پذیری ارتباط (Connectivity):** تست `ping` بین آدرس‌های `10.0.0.2` و `10.0.0.3` با موفقیت انجام شد. تأخیر بسیار پایین (کمتر از 0.1 میلی ثانیه) نشان‌دهنده کارایی بالای پل مجازی لینوکس در انتقال بسته‌ها است.

```
o navid@navid:~/uni/TS/OS/Project/zocker_project/src$ sudo python3 zocker-proxy.py

--- Zocker Proxy CLI (Final Version) ---
Commands: nodes, run, ps, stop, rm, exec, exit
zocker-proxy > [*] Discovery service started on port 8888...

[+] Node Registered/Updated: CID 4
nodes
CID      Total RAM (MB)  Allocated RAM (MB)  Free RAM (MB)
-----
4        14855           0                   14855
zocker-proxy > ps -a
VM      ID          STATUS    PID    AGE    EXIT
-----
4       953f4c7f    running   5330   133m  18s   -
4       c865a86b    running   7498   5m    9s    -
4       0ebede81    running   4980   143m  25s   -
4       673a78f4    running   4555   152m  8s    -
4       2958bf2c    running   7683   4m    55s   -
4       e3f1772c    stopped   0      125m  41s   0
zocker-proxy > connect 4 c865a86b 2958bf2c
Result: Success: Containers connected on 10.0.0.2 <-> 10.0.0.3
zocker-proxy > exec 4 c865a86b ping -c 10.0.0.3
--- Output from c865a86b ---
Error (Code 1): ping: invalid argument: '10.0.0.3'
zocker-proxy > exec 4 c865a86b ping -c 4 10.0.0.3
--- Output from c865a86b ---
PING 10.0.0.3 (10.0.0.3) 56(84) bytes of data.
64 bytes from 10.0.0.3: icmp_seq=1 ttl=64 time=0.115 ms
64 bytes from 10.0.0.3: icmp_seq=2 ttl=64 time=0.084 ms
64 bytes from 10.0.0.3: icmp_seq=3 ttl=64 time=0.072 ms
64 bytes from 10.0.0.3: icmp_seq=4 ttl=64 time=0.086 ms

--- 10.0.0.3 ping statistics ---
4 packets transmitted, 4 received, 0% packet loss, time 3057ms
rtt min/avg/max/mdev = 0.072/0.089/0.115/0.015 ms
zocker-proxy > exec 4 2958bf2c ip addr
--- Output from 2958bf2c ---
1: lo: <LOOPBACK> mtu 65536 qdisc noop state DOWN group default qlen 1000
    link/loopback 00:00:00:00:00:00 brd 00:00:00:00:00:00
6: eth0@if7: <BROADCAST,MULTICAST,UP,LOWER_UP> mtu 1500 qdisc noqueue state UP group default qlen 1000
    link/ether aa:27:c0:6b:50:0b brd ff:ff:ff:ff:ff:ff link-netnsid 0
    inet 10.0.0.3/24 scope global eth0
        valid_lft forever preferred_lft forever
    inet6 fe80::a827:c0ff:fe6b:500b/64 scope link proto kernel_ll
        valid_lft forever preferred_lft forever
```

شکل ۳-۶: تبادل موفق بسته‌های شبکه بین دو کانتینر ایزوله از طریق پل مجازی

۴-۶ جمع‌بندی نهایی

نتایج حاصل از تست‌های عملیاتی نشان می‌دهد که معماری لایه‌بندی شده Zocker توانسته است ایزولاسیون سخت‌گیرانه هسته لینوکس را با قابلیت مدیریت توزیع‌شده ادغام کند. پایداری کانتینرها به عنوان سرویس‌های Systemd و برقراری ارتباط شبکه در بستر VSOCK، موفقیت پیاده‌سازی این پلتفرم را تایید می‌کند.

فصل ۷

چالش‌ها و راه‌حل‌ها

۷-۰-۱ چالش‌های مدیریت کانتینرها

در پیاده‌سازی این پروژه، یکی از چالش‌های اصلی تطبیق PIDها بود. از آنجایی که سرویس سیستم‌دی باید پروسه والد را تحت نظر بگیرد، اما پروسه اصلی ایزولاسیون در فرزند (Child) رخ می‌دهد، مکانیزمی طراحی شد که والد تا زمان پایان کار فرزند منتظر بماند. این کار باعث می‌شود وضعیت سرویس در سیستم‌دی همواره با وضعیت واقعی کانتینر همگام (Sync) باشد.

۷-۰-۲ چالش‌های Zocker Proxy

یکی از چالش‌های دیگر مربوط به راه‌اندازی بخش Zocker Proxy بر روی ماشین مبتنی بر سیستم‌عامل ویندوز بود. در طراحی اولیه، استفاده از مکانیزم ارتباطی VSOCK برای ارتباط بین اجزای ایزوله‌شده در نظر گرفته شده بود. با این حال، به دلیل عدم پشتیبانی بومی VSOCK در ویندوز، شناسه‌های ارتباطی CID به‌درستی منتقل نمی‌شدند و ارتباط پایدار برقرار نمی‌گردید. برای رفع این محدودیت و تضمین عملکرد صحیح مکانیزم ارتباطی، سیستم‌عامل میزبان به لینوکس تغییر داده شد که این امکان استفاده کامل از VSOCK و برقراری ارتباط پایدار بین اجزای سیستم را فراهم کرد.

۷-۰-۳ چالش ایزولاسیون کامل و مدیریت مسیرهای مطلق (Absolute Paths)

در فرآیند پیاده‌سازی، استفاده از فضاها (Namespaces) مستلزم اجرای اسکریپت با دسترسی root بود. این موضوع چالش جدیدی ایجاد کرد؛ چون متغیرهای محیطی سیستم تغییر یافته و تمامی متادیتای کانتینرها به جای پوشه کاربر، به اشتباه در مسیر /root/ ذخیره می‌شدند. همچنین اجرای ابزار مدیریت کانتینر از دایرکتوری‌های مختلف، باعث می‌شد مسیرهای نسبی به درستی شناسایی نشوند.

برای رفع این مشکل، تابعی تحت عنوان get_base_path طراحی شد که مسیر خانه کاربر اصلی را به صورت هوشمند تشخیص داده و یک مسیر مطلق (Absolute Path) در محدوده /zocker/ ایجاد می‌کند. این رویکرد تضمین کرد که فارغ از دایرکتوری جاری ترمینال یا سطح دسترسی اجرایی، تمامی فایل‌های وضعیت و تنظیمات همواره در جایگاه استاندارد خود باقی بمانند.

۷-۰-۴ تداخل مدیریت حافظه در محیط‌های مجازی (WSL2/VM)

یکی از پیچیده‌ترین چالش‌های فنی در لایه هسته، بروز خطای Cannot allocate memory در لحظه‌ی الحاق پردازنده به Cgroups v2 بود. در محیط‌های مجازی نظیر WSL2، زمانی که پردازنده فرزند خود را به گروه کنترل متصل می‌کرد، هسته لینوکس در زمان اجرای فراخوان execvp برای بارگذاری تصویر جدید پردازنده (bash)، به اشتباه با خطای تخصیص حافظه مواجه می‌شد. جهت حل این چالش، ترتیب عملیات اجرایی در ماژول core_engine مورد بازنگری دقیق قرار گرفت. با انتقال عملیات attach به آخرین لحظه ممکن پیش از تغییر تصویر پردازنده و پس از اتمام تمامی تنظیمات فضاها، تداخل در تخصیص منابع مرتفع گشته و پایداری سیستم در محیط‌های مجازی تضمین گردید.

۷-۰-۵ ایزولاسیون فایل سیستم مجازی و مکانیزم نقاط مونت

حتی پس از انجام عملیات chroot، کانتینر همچنان به برخی از نقاط مونت سیستم میزبان دسترسی داشت که یک ریسک امنیتی جدی محسوب می‌شد. همچنین، برای عملکرد صحیح دستورات پایش سیستم نظیر ps، نیاز بود که فایل سیستم مجازی /proc به صورت اختصاصی برای کانتینر در دسترس باشد.

برای دستیابی به ایزولاسیون کامل، از پرچم‌های MS_PRIVATE و MS_REC در مدیریت نقاط

اتصال استفاده شد تا از نشت تغییرات کانتینر به میزبان جلوگیری شود. با پیاده‌سازی مکانیزم سوار کردن (Remount) مجدد دایرکتوری‌های /proc و /sys بلافاصله پس از ورود به محیط ریشه جدید، محیطی کاملاً مستقل فراهم شد که در آن فرآیندها تنها دید محدودی به منابع اختصاص یافته خود دارند.

۷-۰-۶ هماهنگی سطح دسترسی در تعاملات بین‌گروهی

از آنجایی که هسته اجرایی پروژه با دسترسی root فایل وضعیت کانتینر (state.json) را ایجاد می‌کرد، سایر ماژول‌ها نظیر ایجنت که با سطح دسترسی کاربر عادی اجرا می‌شدند، در خواندن و پایش وضعیت کانتینر با خطای عدم دسترسی مواجه می‌شدند. برای رفع این چالش و تسهیل تعامل بین اجزای مختلف سیستم، مکانیزمی در کد هسته تعبیه شد که بلافاصله پس از ایجاد فایل وضعیت، سطح دسترسی آن را به صورت داینامیک به 0o666 تغییر می‌دهد. این راهکار اجازه داد تا لایه‌ی ارکستراسیون بتواند بدون تداخل، اطلاعات حیاتی کانتینر را جهت محاسبات توازن بار استخراج نماید.

پیوست: کدهای منبع پروژه

zocker-proxy.py

```
1 import socket
2 import json
3 import threading
4 import uuid
5 import argparse
6 import shlex
7 import time
8
9 REG_PORT = 8888
10 CMD_PORT = 9999
11 HOST_CID = socket.VMADDR_CID_ANY
12
13 class ZockerProxy:
14     def __init__(self):
15         self.nodes = {} # {cid: {total_mem, alloc_mem, total_cpu...}}
16         self.lock = threading.Lock()
17
18     def start_discovery(self):
19         sock = socket.socket(socket.AF_VSOCK, socket.SOCK_STREAM)
20         sock.bind((HOST_CID, REG_PORT))
21         sock.listen(10)
22         print(f"[*] Discovery service started on port {REG_PORT}...")
23         while True:
24             conn, addr = sock.accept()
25             try:
26                 data = conn.recv(2048).decode('utf-8')
27                 if data:
28                     with self.lock:
29                         self.nodes[addr[0]] = json.loads(data)
30                         print(f"\n[+] Node Registered/Updated: CID {addr[0]}")
31                     conn.sendall(b"ACK")
32             except Exception as e:
33                 print(f"Discovery Error: {e}")
34             finally:
35                 conn.close()
36
37     def send_command(self, cid, payload):
38         sock = socket.socket(socket.AF_VSOCK, socket.SOCK_STREAM)
39         try:
40             sock.settimeout(5)
41             sock.connect((cid, CMD_PORT))
42             sock.sendall(json.dumps(payload).encode('utf-8'))
43             return sock.recv(16384).decode('utf-8')
```

```

44     except Exception as e:
45         return f"Error: {e}"
46     finally:
47         sock.close()
48
49     def select_node(self, required_mem):
50         best_cid = -1
51         min_alloc = float('inf')
52
53         with self.lock:
54             for cid, stats in self.nodes.items():
55                 free_mem = stats['total_mem'] - stats['alloc_mem']
56                 if free_mem >= required_mem:
57                     if stats['alloc_mem'] < min_alloc:
58                         min_alloc = stats['alloc_mem']
59                         best_cid = cid
60         return best_cid
61
62     def update_node_mem(self, cid, mem_change):
63         with self.lock:
64             if cid in self.nodes:
65                 self.nodes[cid]['alloc_mem'] += mem_change
66
67 proxy = ZockerProxy()
68
69 def main():
70     threading.Thread(target=proxy.start_discovery, daemon=True).start()
71
72     print("\n--- Zocker Proxy CLI (Final Version) ---")
73     print("Commands: nodes, run, ps, stop, rm, exec, exit")
74
75     while True:
76         try:
77             line = input("zocker-proxy > ").strip()
78             if not line: continue
79             parts = shlex.split(line)
80             cmd = parts[0]
81
82             if cmd == "nodes":
83                 print(f"{'CID':<10} {'Total RAM (MB)':<15} {'Allocated RAM (MB)':<18} {'Free RAM (MB)'}")
84                 print("-" * 65)
85                 for cid, stats in proxy.nodes.items():
86                     free = stats['total_mem'] - stats['alloc_mem']
87                     print(f"{cid:<10} {stats['total_mem']:<15} {stats['alloc_mem']:<18} {free}")
88
89             elif cmd == "run":
90                 p = argparse.ArgumentParser()
91                 p.add_argument("--memory", type=int, default=512)
92                 p.add_argument("config")
93                 try:
94                     args = p.parse_args(parts[1:])
95                 except: continue
96
97                 if not os.path.exists(args.config):
98                     print(f"Error: File {args.config} not found.")
99                     continue
100
101                 with open(args.config, 'r') as f:
102                     config_data = json.load(f)
103
104                 best_cid = proxy.select_node(args.memory)

```

```

105
106     if best_cid == -1:
107         print("Error: No node has enough free RAM for this request.")
108         continue
109
110     u_id = str(uuid.uuid4())[:8]
111     print(f"[*] Dispatching container {u_id} to VM {best_cid}...")
112
113     res = proxy.send_command(best_cid, {
114         "cmd": "run", "uuid": u_id, "config": config_data,
115         "limits": {"memory": args.memory}
116     })
117
118     print(f"Result: {res}")
119
120     if "OK" in res:
121         proxy.update_node_mem(best_cid, args.memory)
122
123     elif cmd == "ps":
124         header = f"{'VM':<5} {'ID':<12} {'STATUS':<10} {'PID':<8} {'AGE':<12} {'EXIT'}"
125         print(header)
126         print("-" * len(header))
127         for cid in list(proxy.nodes.keys()):
128             res = proxy.send_command(cid, {"cmd": "ps"})
129             try:
130                 containers = json.loads(res)
131                 for c in containers:
132                     exit_status = c.get('exit_code', '-') if c['status'] == 'stopped' else '-'
133                     print(f"{cid:<5} {c['id'][:8]:<12} {c['status']:<10} {c['pid']:<8} {c.get('age', '-')<12} {exit_status}")
134             except:
135                 print(f"[{cid}] Could not fetch PS data.")
136
137     elif cmd == "stop":
138         if len(parts) < 3:
139             print("Usage: stop <VM_CID> <UUID>")
140             continue
141         cid, u_id = int(parts[1]), parts[2]
142         print(proxy.send_command(cid, {"cmd": "stop", "uuid": u_id}))
143
144     elif cmd == "rm":
145         # rm <VM_CID> <UUID> [-f]
146         if len(parts) < 3:
147             print("Usage: rm <VM_CID> <UUID> [-f]")
148             continue
149         cid = int(parts[1])
150         u_id = parts[2]
151         force = "-f" in parts
152
153         res = proxy.send_command(cid, {"cmd": "rm", "uuid": u_id, "force": force})
154         print(f"Result from VM {cid}: {res}")
155
156     elif cmd == "exec":
157         if len(parts) < 4:
158             print("Usage: exec <VM_CID> <UUID> <CMD>")
159             continue
160         cid, u_id = int(parts[1]), parts[2]
161         command = " ".join(parts[3:])
162         res = proxy.send_command(cid, {"cmd": "exec", "uuid": u_id, "shell_cmd": command})
163         print(f"--- Output from {u_id} ---\n{res}")
164

```

```
165
166         elif cmd == "exit":
167             break
168     except Exception as e:
169         print(f"CLI Error: {e}")
170
171 if __name__ == "__main__":
172     import os
173     main()
```

zocker-agent.py

```
1 import socket
2 import json
3 import os
4 import psutil
5 import time
6 import threading
7 import manager
8
9 HOST_CID = 2
10 REG_PORT = 8888
11 CMD_PORT = 9999
12
13 def get_stats():
14     total_m = psutil.virtual_memory().total // (1024 * 1024)
15     total_c = psutil.cpu_count()
16     states = manager.get_all_container_states()
17     alloc_m = 0
18     for s in states:
19         if s.get('status') == 'running':
20             alloc_m += s.get('limits', {}).get('memory', 0)
21     return {
22         "total_mem": total_m,
23         "alloc_mem": alloc_m,
24         "total_cpu": total_c,
25         "alloc_cpu": 0
26     }
27
28 def listen():
29     server = socket.socket(socket.AF_VSOCK, socket.SOCK_STREAM)
30     server.bind((socket.VMADDR_CID_ANY, CMD_PORT))
31     server.listen(5)
32     print(f"[*] Agent is listening for commands on VSOCK port {CMD_PORT}...")
33
34     while True:
35         conn, addr = server.accept()
36         client_cid = addr[0]
37         try:
38             raw = conn.recv(16384).decode('utf-8')
39             req = json.loads(raw)
40             cmd = req.get('cmd')
41
42             print(f"[#] Received command: '{cmd}' from Host (CID {client_cid})")
43
44             if cmd == "get_stats":
45                 conn.sendall(json.dumps(get_stats()).encode('utf-8'))
46
47             elif cmd == "run":
48                 u_id = req['uuid']
49                 print(f"[*] Starting container creation: {u_id[:12]}")
50
51                 c_dir = os.path.join(manager.BASE_PATH, u_id)
52                 os.makedirs(c_dir, exist_ok=True)
53
54                 state = {
55                     "id": u_id, "status": "creating", "pid": 0,
56                     "bundle": c_dir, "limits": req['limits'],
57                     "annotations": {"name": f"zocker-{u_id[:4]}"}
58                 }
59                 with open(os.path.join(c_dir, "config.json"), 'w') as f: json.dump(req['config'], f)
```

```

60         with open(os.path.join(c_dir, "state.json"), 'w') as f: json.dump(state, f)
61
62         manager.create_and_start_service(u_id)
63         print(f"[+] Container {u_id[:12]} service started successfully.")
64         conn.sendall(b"OK: Service Started")
65
66         elif cmd == "ps":
67             all_states = manager.get_all_container_states()
68             now = time.time()
69             for s in all_states:
70                 created_at = s.get('created_at') or now
71                 age_seconds = int(now - created_at)
72
73                 if age_seconds < 60: s['age'] = f"{age_seconds}s"
74                 else: s['age'] = f"{age_seconds // 60}m {age_seconds % 60}s"
75
76             conn.sendall(json.dumps(all_states).encode('utf-8'))
77
78         elif cmd == "stop":
79             u_id = req.get('uuid')
80             print(f"[*] Stopping container: {u_id[:12]}")
81             os.system(f"sudo systemctl stop zocker-{u_id}")
82             conn.sendall(b"OK: Container Stopped")
83
84         elif cmd == "rm":
85             u_id = req.get('uuid')
86             print(f"[*] Removing container files: {u_id[:12]}")
87             result = manager.remove_container(u_id, req.get('force', False))
88             conn.sendall(str(result).encode('utf-8'))
89
90         elif cmd == "exec":
91             u_id = req.get('uuid')
92             shell_cmd = req.get('shell_cmd')
93             print(f"[*] Executing command inside {u_id[:12]}: {shell_cmd}")
94             result = manager.exec_in_container_V2(u_id, shell_cmd)
95             conn.sendall(str(result).encode('utf-8'))
96
97     except Exception as e:
98         print(f"[!] Error during command execution: {str(e)}")
99         conn.sendall(f"Error: {str(e)}.encode('utf-8')")
100     finally:
101         conn.close()
102
103 if __name__ == "__main__":
104     print("--- Zocker Agent Starting ---")
105     threading.Thread(target=listen, daemon=True).start()
106
107     print(f"[*] Trying to register at Proxy (Host CID {HOST_CID})...")
108     while True:
109         try:
110             s = socket.socket(socket.AF_VSOCK, socket.SOCK_STREAM)
111             s.connect((HOST_CID, REG_PORT))
112             s.sendall(json.dumps(get_stats()).encode('utf-8'))
113             if s.recv(1024) == b"ACK":
114                 print("[+] Successfully registered at Proxy!")
115                 break
116         except:
117             time.sleep(2)
118
119     while True:
120         time.sleep(1)

```


zocker.py

```
1 import argparse
2 import sys
3 import uuid
4 import json
5 import os
6 import time
7 import manager
8
9 def format_age(timestamp):
10     if not timestamp:
11         return "-"
12     diff = int(time.time() - timestamp)
13     if diff < 60: return f"{diff}s"
14     elif diff < 3600: return f"{diff // 60}m"
15     elif diff < 86400: return f"{diff // 3600}h"
16     else: return f"{diff // 86400}d"
17
18 def main():
19     parser = argparse.ArgumentParser(description="Zocker Local Management CLI (Part 1)")
20     subparsers = parser.add_subparsers(dest="command")
21
22     # 1.run
23     run_parser = subparsers.add_parser("run")
24     run_parser.add_argument("--memory", type=int, default=512)
25     run_parser.add_argument("--cpu", type=int, default=1024)
26     run_parser.add_argument("--name", help="Custom name for the container", default=None)
27     run_parser.add_argument("config", help="Path to config.json")
28
29     # 2.ps
30     ps_parser = subparsers.add_parser("ps")
31     ps_parser.add_argument("-a", action="store_true", help="Show all containers")
32
33     # 3.exec
34     exec_parser = subparsers.add_parser("exec")
35     exec_parser.add_argument("id", help="Container UUID")
36     exec_parser.add_argument("cmd", help="Command to run")
37
38     # 4.stop
39     stop_parser = subparsers.add_parser("stop")
40     stop_parser.add_argument("id")
41
42     # 5.restart
43     restart_parser = subparsers.add_parser("restart")
44     restart_parser.add_argument("id")
45
46     # 6.rm
47     rm_parser = subparsers.add_parser("rm")
48     rm_parser.add_argument("id")
49     rm_parser.add_argument("-f", action="store_true", help="Force remove")
50
51     args = parser.parse_args()
52
53     if args.command == "run":
54         u_id = args.name if args.name else str(uuid.uuid4())[:8]
55         c_dir = os.path.join(manager.BASE_PATH, u_id)
56
57         try:
58             with open(args.config, 'r') as f:
59                 new_config_data = json.load(f)
```

```

60     except Exception as e:
61         print(f"Error reading config: {e}")
62         return
63
64     if os.path.exists(c_dir):
65         old_config_path = os.path.join(c_dir, "config.json")
66         if os.path.exists(old_config_path):
67             with open(old_config_path, 'r') as f:
68                 old_config_data = json.load(f)
69
70             if old_config_data == new_config_data:
71                 print(f"[-] Container '{u_id}' is already running with this exact config.")
72                 return
73
74             print(f"[*] Config change detected for '{u_id}'. Restarting...")
75
76             manager.remove_container(u_id, force=True)
77             time.sleep(1)
78
79             os.makedirs(c_dir, exist_ok=True)
80             with open(os.path.join(c_dir, "config.json"), 'w') as f:
81                 json.dump(new_config_data, f, indent=4)
82
83             manager.create_and_start_service(u_id)
84             print(f"[+] Container {u_id} started.")
85
86 elif args.command == "ps":
87     containers = manager.get_all_container_states()
88     print(f"{'CONTAINER ID':<15} {'STATUS':<12} {'PID':<10} {'EXIT':<8} {'AGE / FINISHED'}")
89     print("-" * 65)
90
91     for c in containers:
92         status = c.get('status', 'unknown')
93         if not args.a and status != 'running': continue
94
95         c_id = c.get('id', 'unknown')[:12]
96         pid = c.get('pid', 0) if status == 'running' else 0
97         exit_code = c.get('exit_code', '-') if status == 'stopped' else '-'
98
99         if status == 'running':
100             age_val = format_age(c.get('created_at'))
101         else:
102             finished_at = c.get('finished_at')
103             age_val = f"Exited ({format_age(finished_at)} ago)" if finished_at else "unknown"
104
105         print(f"{{c_id:<15}} {{status:<12}} {{pid:<10}} {{exit_code:<8}} {{age_val}}")
106
107 elif args.command == "exec":
108     print(f"[*] Entering container {args.id}...")
109     manager.exec_in_container(args.id, args.cmd)
110
111 elif args.command == "stop":
112     os.system(f"sudo systemctl stop zocker-{args.id}")
113     print(f"[+] Container {args.id} stopped.")
114
115 elif args.command == "restart":
116     print(f"[*] Restarting container {args.id}...")
117     os.system(f"sudo systemctl stop zocker-{args.id}")
118
119     time.sleep(1)
120

```

```
121     os.system(f"sudo systemctl start zocker-{args.id}")
122
123     print(f"[+] Container {args.id} restarted successfully.")
124
125     elif args.command == "rm":
126         success, msg = manager.remove_container(args.id, args.f)
127         print(msg)
128
129 if __name__ == "__main__":
130     main()
```

core_engine.py

```
1 import os
2 import sys
3 import json
4 import ctypes
5 import time
6 from pathlib import Path
7 from resources import ResourceManager, get_limits_from_config
8 from isolation import apply_isolation, set_container_hostname
9
10 def get_base_path():
11     current_dir = os.path.dirname(os.path.abspath(__file__))
12     parts = current_dir.split(os.sep)
13     if 'home' in parts:
14         home_index = parts.index('home')
15         user_home = os.sep + os.path.join(parts[home_index], parts[home_index + 1])
16         return os.path.join(user_home, ".zocker", "containers")
17     return os.path.expanduser("~/zocker/containers")
18
19 BASE_PATH = get_base_path()
20
21 # --- Flags ---
22 CLONE_NEWPID = 0x20000000
23 CLONE_NEWNS = 0x00020000
24 CLONE_NEWUTS = 0x04000000
25 MS_REC = 16384
26 MS_PRIVATE = 262144
27
28 def update_state_file(container_id, pid, status, exit_code=0):
29     state_path = os.path.join(BASE_PATH, container_id, "state.json")
30     created_at = time.time()
31     if os.path.exists(state_path):
32         with open(state_path, 'r') as f:
33             try:
34                 old_data = json.load(f)
35                 created_at = old_data.get("created_at", time.time())
36             except: pass
37
38     state_data = {
39         "id": container_id,
40         "status": status,
41         "pid": pid,
42         "bundle": os.path.dirname(state_path),
43         "created_at": created_at,
44         "exit_code": exit_code,
45         "finished_at": time.time() if status == "stopped" else None
46     }
47     os.makedirs(os.path.dirname(state_path), exist_ok=True)
48     with open(state_path, 'w') as f:
49         json.dump(state_data, f)
50
51 def manage_rootfs(rootfs_path, action="setup"):
52     dirs = ['bin', 'lib', 'lib64', 'usr', 'etc', 'proc', 'sys', 'dev']
53
54     if action == "setup":
55         for d in dirs:
56             target = os.path.join(rootfs_path, d)
57             os.makedirs(target, exist_ok=True)
58             source = f"/{d}"
```

```

59         if os.path.exists(source):
60             os.system(f"mount --bind -n {source} {target}")
61             os.system(f"mount -o remount,ro,bind {target}")
62
63     elif action == "cleanup":
64         os.system(f"umount -fl {rootfs_path}/* > /dev/null 2>&1")
65
66         for d in reversed(dirs):
67             target = os.path.join(rootfs_path, d)
68             if os.path.ismount(target):
69                 os.system(f"umount -fl {target}")
70
71 def run_container_process(container_id, rootfs_path, config_data, rm):
72     try:
73         libc = ctypes.CDLL("libc.so.6")
74         libc.mount(None, b"/", None, MS_REC | MS_PRIVATE, None)
75         if "mounts" in config_data:
76             for mnt in config_data["mounts"]:
77                 src = mnt["source"]
78                 dst = os.path.join(rootfs_path, mnt["destination"].lstrip("/"))
79                 os.makedirs(dst, exist_ok=True)
80                 os.system(f"mount --bind {src} {dst}")
81                 print(f"[*] Mounted {src} to {mnt['destination']}")
82
83             os.system(f"mount --bind /dev {os.path.join(rootfs_path, 'dev')}")
84             os.chroot(rootfs_path)
85             os.chdir("/")
86
87             os.makedirs("/proc", exist_ok=True)
88             os.system("mount -t proc proc /proc")
89             os.makedirs("/sys", exist_ok=True)
90             os.system("mount -t sysfs sys /sys")
91
92             if "env" in config_data:
93                 for env_item in config_data["env"]:
94                     if "=" in env_item:
95                         k, v = env_item.split("=", 1)
96                         os.environ[k] = v
97
98             set_container_hostname(f"zocker-{container_id[:6]}")
99             rm.attach(os.getpid())
100
101             os.environ["PATH"] = "/usr/local/sbin:/usr/local/bin:/usr/sbin:/usr/bin:/sbin:/bin"
102             os.environ["TERM"] = "xterm"
103
104             print(f"[*] Container {container_id} is starting as PID 1...")
105
106             os.execlp("sh", "sh", "-c", "tail -f /dev/null")
107
108     except Exception as e:
109         with open("/tmp/zocker_error.log", "a") as f:
110             f.write(f"Error in child: {str(e)}\n")
111         sys.exit(1)
112
113
114 def start_container(container_id):
115     try:
116         base_dir = os.path.join(BASE_PATH, container_id)
117         config_path = os.path.join(base_dir, "config.json")
118
119         with open(config_path, 'r') as f:

```

```

120     config_data = json.load(f)
121
122     rootfs_path = os.path.join(base_dir, "rootfs")
123     mem, cpu = get_limits_from_config(container_id)
124     rm = ResourceManager(container_id)
125     rm.create_limits(mem, cpu)
126
127     manage_rootfs(rootfs_path, "setup")
128     apply_isolation()
129
130     libc = ctypes.CDLL("libc.so.6")
131     if libc.unshare(CLONE_NEWPID | CLONE_NEWNS | CLONE_NEWUTS) != 0:
132         raise OSError("Unshare failed")
133
134     pid = os.fork()
135     if pid == 0:
136         run_container_process(container_id, rootfs_path, config_data, rm)
137     else:
138         update_state_file(container_id, pid, "running")
139         try:
140             _, status = os.waitpid(pid, 0)
141             exit_code = os.waitstatus_to_exitcode(status)
142         except:
143             exit_code = 0
144
145         update_state_file(container_id, 0, "stopped", exit_code)
146         manage_rootfs(rootfs_path, "cleanup")
147     except Exception as e:
148         print(f"Error: {e}")
149         sys.exit(1)
150
151 if __name__ == "__main__":
152     if len(sys.argv) > 1:
153         start_container(sys.argv[1])

```

manager.py

```
1 import os
2 import json
3 import shutil
4 from pathlib import Path
5 import time
6 import subprocess
7 import core_engine
8
9 def get_base_path():
10     current_dir = os.path.dirname(os.path.abspath(__file__))
11     parts = current_dir.split(os.sep)
12     if 'home' in parts:
13         home_index = parts.index('home')
14         user_home = os.sep + os.path.join(parts[home_index], parts[home_index + 1])
15         return os.path.join(user_home, ".zocker", "containers")
16     return os.path.expanduser("~/zocker/containers")
17
18 BASE_PATH = get_base_path()
19 SYSTEMD_PATH = "/etc/systemd/system"
20
21
22 def remove_container(container_id, force=False):
23     container_dir = os.path.join(BASE_PATH, container_id)
24     state_path = os.path.join(container_dir, "state.json")
25     rootfs_path = os.path.join(container_dir, "rootfs")
26
27     if os.path.exists(state_path):
28         with open(state_path, 'r') as f:
29             state = json.load(f)
30             if state['status'] == 'running' and not force:
31                 return False, "Error: Container is running. Use -f to force remove."
32
33     print(f"[*] Stopping service zocker-{container_id}...")
34     os.system(f"sudo systemctl stop zocker-{container_id} > /dev/null 2>&1")
35
36     service_link = os.path.join(SYSTEMD_PATH, f"zocker-{container_id}.service")
37     if os.path.exists(service_link):
38         os.system(f"sudo rm {service_link}")
39         os.system("sudo systemctl daemon-reload")
40
41     print(f"[*] Unmounting rootfs for {container_id}...")
42     core_engine.manage_rootfs(rootfs_path, action="cleanup")
43
44     time.sleep(1)
45
46     if os.path.exists(container_dir):
47         exit_code = os.system(f"sudo rm -rf {container_dir}")
48         if exit_code == 0:
49             return True, "Container removed successfully."
50         else:
51             return False, "Failed to remove directory even with sudo rm -rf."
52
53
54 def exec_in_container(container_id, command):
55     base_dir = os.path.join(BASE_PATH, container_id)
56     state_path = os.path.join(base_dir, "state.json")
57     rootfs_path = os.path.join(base_dir, "rootfs")
58
59     if not os.path.exists(state_path):
```

```

60     print("Error: Container state not found.")
61     return
62     with open(state_path, 'r') as f:
63         state = json.load(f)
64     pid = state.get('pid')
65
66     if state['status'] != 'running':
67         print("Error: Container is not running.")
68         return
69     ns_cmd = f"sudo nsenter -t {pid} -m -u -i -n -p chroot {rootfs_path} {command}"
70
71     os.system(ns_cmd)
72
73 def exec_in_container_V2(container_id, command):
74     container_dir = os.path.join(BASE_PATH, container_id)
75     state_path = os.path.join(container_dir, "state.json")
76
77     with open(state_path, 'r') as f:
78         state = json.load(f)
79         pid = state.get('pid')
80
81     rootfs_path = os.path.join(container_dir, "rootfs")
82     ns_cmd = f"sudo nsenter -t {pid} -m -u -i -n -p chroot {rootfs_path} {command} 2>&1"
83
84     try:
85         output = subprocess.check_output(ns_cmd, shell=True, stderr=subprocess.STDOUT)
86         return output.decode('utf-8').strip()
87     except subprocess.CalledProcessError as e:
88         return f"Error (Code {e.returncode}): {e.output.decode('utf-8').strip()}"
89     except Exception as e:
90         return f"Manager Error: {str(e)}"
91
92 def create_and_start_service(container_id):
93     current_dir = os.path.dirname(os.path.abspath(__file__))
94     exec_path = os.path.join(current_dir, "core_engine.py")
95     container_dir = os.path.join(BASE_PATH, container_id)
96     state_file_path = os.path.join(container_dir, "state.json")
97
98     home_dir = os.path.abspath(os.path.join(BASE_PATH, "..", ".."))
99
100     stop_post_cmd = (
101         f"/usr/bin/python3 -c "
102         f"\nimport json, time; "
103         f"p='{state_file_path}'; "
104         f"s=json.load(open(p)); "
105         f"s.update({'status':'stopped', 'pid':0, 'finished_at':time.time()}); "
106         f"json.dump(s, open(p,'w'))\n"
107     )
108
109     content = f"""[Unit]
110 Description=Zocker Container {container_id}
111 After=network.target
112
113 [Service]
114 Type=simple
115 Environment=HOME={home_dir}
116 ExecStart=/usr/bin/python3 {exec_path} {container_id}
117 ExecStopPost={stop_post_cmd}
118 Restart=on-failure
119 RestartSec=3
120 User=root

```



```

121 WorkingDirectory={current_dir}
122
123 [Install]
124 WantedBy=multi-user.target
125 """
126     service_file = os.path.join(container_dir, f"zocker-{container_id}.service")
127     os.makedirs(container_dir, exist_ok=True)
128     with open(service_file, "w") as f:
129         f.write(content)
130
131     target_link = os.path.join(SYSTEMD_PATH, f"zocker-{container_id}.service")
132     os.system(f"sudo ln -sf {service_file} {target_link}")
133     os.system("sudo systemctl daemon-reload")
134     os.system(f"sudo systemctl start zocker-{container_id}")
135
136 def get_all_container_states():
137     containers = []
138     if not os.path.exists(BASE_PATH): return containers
139     for c_id in os.listdir(BASE_PATH):
140         p = os.path.join(BASE_PATH, c_id, "state.json")
141         if os.path.exists(p):
142             with open(p, 'r') as f:
143                 try: containers.append(json.load(f))
144                 except: pass
145     return containers

```

register.py

```
1 import socket
2 import json
3 import psutil
4 import platform
5
6 def get_system_specs():
7     specs = {
8         "node_name": socket.gethostname(),
9         "ip_address": socket.gethostbyname(socket.gethostname()),
10        "os": f"{platform.system()} {platform.release()}",
11        "total_ram_gb": round(psutil.virtual_memory().total / (1024**3), 2),
12        "available_ram_gb": round(psutil.virtual_memory().available / (1024**3), 2),
13        "cpu_cores": psutil.cpu_count(logical=True),
14        "status": "ready"
15    }
16    return specs
17
18 def register_node(server_ip, server_port):
19     data = get_system_specs()
20     try:
21         with socket.socket(socket.AF_INET, socket.SOCK_STREAM) as s:
22             s.settimeout(5)
23             s.connect((server_ip, server_port))
24             s.sendall(json.dumps(data).encode('utf-8'))
25             print(f"Success: {data['total_ram_gb']}GB RAM reported.")
26     except Exception as e:
27         print(f"Error: {e}")
28
29 if __name__ == "__main__":
30     DEST_IP = "192.168.1.104"
31     DEST_PORT = 5000
32     register_node(DEST_IP, DEST_PORT)
```